

# Using SlabHash in Your Project

A practical API guide for owensgroup/SlabHash

---

## What SlabHash gives you

SlabHash is a **header-only CUDA library**: a GPU-resident hash table (`ConcurrentMap`, key→value) or set (`ConcurrentSet`, keys only) that is built, searched, and deleted from via CUDA kernels. There is nothing to link — you `#include` one header and instantiate a template class. Keys and values must be (at most) 32-bit types (`uint32_t`, `int32_t`, `float`, etc.).

## 1. Getting it into your project

SlabHash depends on its own submodule, `SlabAlloc` (the memory allocator). Vendor both as git submodules, or copy their `src/` folders directly into your project:

```
git submodule add https://github.com/owensgroup/SlabHash third_party/SlabHash
cd third_party/SlabHash
git submodule update --init SlabAlloc # pulls in the allocator headers
```

Add both header directories to your include path and `#include` the single entry-point header plus the application-level wrapper:

```
#include "gpu_hash_table.cuh" // pulls in slab_hash.cuh internally
```

In CMake:

```
include_directories(third_party/SlabHash/src)
include_directories(third_party/SlabHash/SlabAlloc/src)
# pick the flag matching your GPU's compute capability, e.g. for an
# Ampere/RTX 30-series card (sm_86) add to your nvcc flags:
set(CUDA_NVCC_FLAGS ${CUDA_NVCC_FLAGS} -gencode=arch=compute_86,code=sm_86)
```

Your `.cu` file that includes `gpu_hash_table.cuh` must be compiled with `nvcc` (it's real CUDA/C++ template code, not a precompiled library) and with `-std=c++11` or newer.

## 2. The class you actually use: `gpu_hash_table`

```
template <typename KeyT, typename ValueT, SlabHashTypeT SlabHashT>
class gpu_hash_table { ... };
```

- `KeyT`, `ValueT`: your key/value types ( $\leq 32$  bits each).
- `SlabHashT`: `SlabHashTypeT::ConcurrentMap` (key→value) or `SlabHashTypeT::ConcurrentSet` (keys only — pass `KeyT` for both template arguments in that case).

Construct one instance per table:

```
gpu_hash_table<uint32_t, uint32_t, SlabHashTypeT::ConcurrentMap>
table(max_keys, // capacity hint used only to size scratch buffers
     num_buckets, // see "choosing num_buckets" below
     device_idx, // which GPU, e.g. 0
     seed, // optional; 0 = time-based random seed
     identity_hash); // optional; false = normal hashing (leave false)
```

This single call `cudaMalloc`s everything the table needs (the bucket array, the allocator's memory pool, and scratch device buffers for keys/values/ queries/results). There is no separate "init" step.

## 3. Choosing `num_buckets`

Each bucket's base slab holds **15 key/value pairs** before it needs to grow a chain via the allocator. Pick `num_buckets` so that, on average, each bucket ends up close to full without needing many overflow slabs:

```
uint32_t num_buckets = (max_keys + 15 - 1) / 15;    // ~1 key per slot
// or, to leave headroom (recommended if key counts vary at runtime):
uint32_t num_buckets = (max_keys + (15 * 0.7) - 1) / (uint32_t)(15 * 0.7);
```

The table does **not** resize its bucket count later — pick `num_buckets` for your expected maximum key count up front. Chains can still grow dynamically beyond that (via the allocator), just less efficiently as load increases.

## 4. Core operations

All operations take/return **host pointers** — `gpu_hash_table` manages the device-side copies (and its own persistent device buffers) internally, and every call blocks until the relevant kernel finishes, returning that kernel's elapsed time in milliseconds (useful for benchmarking; safe to ignore otherwise).

```
std::vector<uint32_t> h_key(N), h_value(N), h_query(N), h_result(N, 0xFFFFFFFFu);

// Insert N key/value pairs. Duplicate keys are allowed and both survive
// as separate entries (see "gotchas" below).
float t_build = table.hash_build(h_key.data(), h_value.data(), N);

// Insert, but silently skip a key if it's already present (first write wins).
float t_build_u = table.hash_build_with_unique_keys(h_key.data(), h_value.data(), N);

// Look up each of h_query[i]; missing keys leave h_result[i] untouched
// (pre-fill h_result with a sentinel, e.g. 0xFFFFFFFF, to detect "not found").
float t_search = table.hash_search(h_query.data(), h_result.data(), Q);

// Delete each key in h_del (a no-op for keys that aren't present).
float t_delete = table.hash_delete(h_del.data(), D);

// Count occurrences of each key in h_query (only meaningful after
// hash_build, which allows duplicates — always 0 or 1 after
// hash_build_with_unique_keys).
float t_count = table.hash_count(h_query.data(), h_count.data(), Q);
```

`ConcurrentSet` supports `hash_build`/`hash_build_with_unique_keys` and `hash_search` (results are the echoed key itself if found, or the not-found sentinel) but **does not** currently implement `hash_delete`/bulk search/load-factor queries — those are stubbed with empty bodies in the shipped code. Use `ConcurrentMap` if you need working delete on a set-like table (store a dummy value and ignore it).

## 5. Minimal end-to-end example

```

#include "gpu_hash_table.cuh"
#include <vector>
#include <iostream>

int main() {
    using KeyT = uint32_t;
    using ValueT = uint32_t;
    const uint32_t N = 1'000'000;
    const uint32_t num_buckets = (N + 14) / 15;    // ~1 key per slot

    std::vector<KeyT> h_key(N);
    std::vector<ValueT> h_value(N);
    for (uint32_t i = 0; i < N; i++) { h_key[i] = i; h_value[i] = i * 2; }

    gpu_hash_table<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>
        table(N, num_buckets, /*device_idx=*/0);

    table.hash_build(h_key.data(), h_value.data(), N);

    std::vector<ValueT> h_result(N, 0xFFFFFFFFu);
    table.hash_search(h_key.data(), h_result.data(), N);    // look up every key

    for (uint32_t i = 0; i < 5; i++)
        std::cout << h_key[i] << " -> " << h_result[i] << "\n";
    return 0;    // table's destructor frees all GPU memory
}

```

Compile with, e.g.:

```

nvcc -std=c++11 -gencode=arch=compute_86,code=sm_86 \
    -I third_party/SlabHash/src -I third_party/SlabHash/SlabAlloc/src \
    main.cu -o main

```

## 6. Gotchas worth knowing before you rely on this in production

- **0xFFFFFFFF is a reserved sentinel.** It marks empty slots internally and is also the default “not found” value for search results — do not use it as a real key or value, or it will be indistinguishable from “empty.”
- **Duplicate keys are allowed by `hash_build`.** If you need set-like uniqueness, use `hash_build_with_unique_keys` instead, or de-duplicate on the host first.
- **The bucket count is fixed at construction.** There’s no rehash/resize — size `num_buckets` for your expected maximum, not your starting size.
- **Deleted slots aren’t returned to the allocator.** A slab that’s had all its keys deleted stays allocated to that bucket’s chain; memory isn’t reclaimed for reuse elsewhere. Long-running processes that insert/delete heavily across many distinct buckets will grow the allocator’s memory footprint over time.
- **This is not thread-safe with the CPU** in the sense of overlapping kernel launches — issue one `hash_*` call at a time per table (they’re synchronous from the host’s point of view already, since each returns a timing value after the kernel completes).
- **Pick your GPU’s actual compute capability flag.** The upstream `CMakeLists.txt` only enables `sm_35` by default; if you vendor the source directly (rather than using their CMake build), make sure your own build passes `-gencode` for your GPU, or the kernels won’t run (or won’t be compiled at all).